Operating Systems Lab — Assignment 7 Report

Manav Singla, 2402MC02 — xv6-riscv — Threads, Semaphores, Deadlock Detection, Priority Scheduling with Aging

Q1 — Kernel-supported threads: `clone()` / `join()`

Design and data structures

- **struct proc** gains `int isthread` (marks a thread) and `uint64 trapva` (user VA where this proc's trapframe is mapped). A thread's `pagetable` pointer is the *same* pointer as its leader's, so the address space is shared, while `kstack`, `trapframe` and `context` stay private.
- **Per-thread trapframes.** Stock xv6-riscv maps every trapframe at one fixed VA, `TRAPFRAME`. Several threads in one page table need different VAs, so thread *i* (proc-table slot) maps its trapframe at `THREADTRAP(i) = TRAPFRAME − (i+1)*PGSIZE`. `trampoline.S` no longer loads the constant `TRAPFRAME`. `prepare_return()` writes `p->trapva` into `sscratch`; `uservec` does `csrrw a0, sscratch, a0` and `userret` does `csrr a0, sscratch`.
- **kclone(fcn, arg, stack)** calls `allocproc()`, throws away the fresh page table, points `np->pagetable` at the caller's, maps `np->trapframe` at `THREADTRAP`, copies the caller's registers and sets `epc = fcn`, `a0 = arg`, `sp = stack + PGSIZE`. Open files and cwd are duplicated. The parent is always the group leader.
- **kjoin()** sleeps on the leader exactly like `wait()`, but only reaps children with `isthread = 1`. `wait()` was changed to skip threads, so the two never steal each other's children.
- **exit / fork / wait / exec.** `freeproc()` of a thread only `uvmunmap`s its own trapframe page and never frees the shared page table. When the leader exits, it first marks all its threads killed and joins them, so the page table is freed only after every thread is gone. `fork()` from any thread copies `0..sz` of the shared space into a new private process. `growproc` / `sbrk` propagate the new `sz` to every member through `setsz()`. `exec()` is refused inside a thread.

Q2 — Counting semaphores and Producer–Consumer

- `struct semaphore { int value; int wakeups; int used; struct spinlock lk; }`. A fixed table of **10** lives in `semtable` together with a table spinlock, which `sem_init` holds.
- `sem_wait`: `value--`; if `value < 0` the caller sleeps on the semaphore (`sleep_prepare` / `sleep` with `lk` released) until a wakeup token exists. `sem_signal`: `value++`; if `value <= 0` it adds a token and calls `wakeup()`. The `wakeups` counter makes the semaphore immune to spurious wakeups, and a killed sleeper backs out, so a leader's exit can always join its threads.
- `synctest` runs the same 3 × 100,000 workload twice: once unprotected, once with `sem_wait(MUTEX)` / `sem_signal(MUTEX)` around `counter = counter + 1`.
- `prodcons`: one producer thread and one consumer thread, buffer of 5, semaphores EMPTY=5, FULL=0, MUTEX=1. The consumer is slow for the first 10 items, so the producer must block on a full buffer. The producer is slow for the last 10, so the consumer must block on an empty one. Logging happens inside MUTEX.

Run (CPUS=3)	Final counter	Lost updates	Time
Q1 `threadtest` (no sync)	102,005	197,995	< 1 tick
Q2 `synctest` phase 1 (no sync)	104,107	195,893	1 tick
Q2 `synctest` phase 2 (semaphore)	**300,000**	**0**	226 ticks
Q2 `prodcons`	20 produced / 20 consumed, max occupancy 5/5, final count 0, 0 overflow/underflow → PASS		

Q3 — Deadlock detection and Banker's avoidance (`deadlock.c`)

- Static arrays `Allocation[n][m]`, `Request[n][m]`, `Max[n][m]`, `Need = Max − Allocation`, `Available[m]`. Every step prints Available, Allocation and Request/Need.
- `detect_deadlock()`: `Work = Available`, and a process holding nothing counts as finished. Repeatedly pick an unfinished process with `Request ≤ Work`, add its Allocation to Work and mark it finished. Processes left unfinished are returned as the deadlocked set.
- `is_safe_state()` is the same loop with Need instead of Request. `request()` runs the Banker's check: `Req ≤ Need`, `Req ≤ Available`, pretend-allocate, test safety, roll back if unsafe.
- **Real deadlock:** process A keeps pipe1's write end and blocks reading pipe2. Process B keeps pipe2's write end and blocks reading pipe1. Each signals readiness through a third pipe. The parent then encodes the situation (R0 = pipe1, R1 = pipe2) and runs `detect_deadlock()`. Afterwards it kills both, and both exit with status −1, which proves they never got past the blocking read.

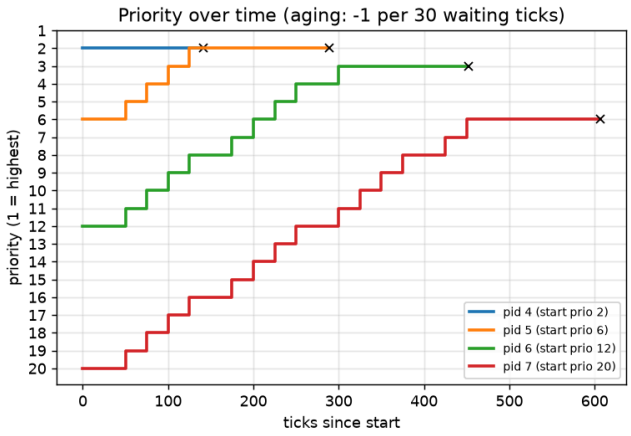
Test	Input	Result
1 Detection	5×3, Available 0 0 0, P2 requests (0,0,1)	P0 finishes → Work 0 1 0, nothing else fits: **DEADLOCK {P1,P2,P3,P4}**
2 Detection	same, P2 requests (0,0,0)	**No deadlock**, order P0 P2 P3 P4 P1
3a Banker's	Avail 3 3 2, classic Max/Alloc	**SAFE** <P1 P3 P4 P0 P2>

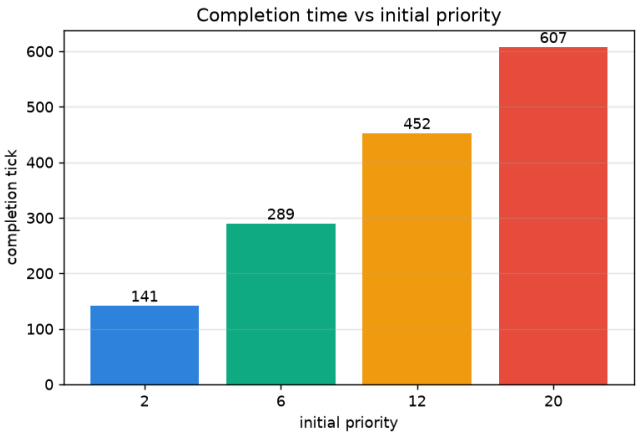
3b Request P1 (1,0,2)	Avail → 2 3 0	**GRANTED**, still safe
3c Request P0 (0,2,0)	Avail would be 2 1 0	No Need fits Work → **UNSAFE, denied and rolled back**
4 Pipes (real)	A holds pipe1 wants pipe2; B holds pipe2 wants pipe1	**DEADLOCK {A,B}**, circular wait A→B→A

Q4 — Priority-based preemptive scheduler with aging

- `struct proc` gains `priority` (1 = highest, 20 = lowest; `allocproc` sets 10) and `waitticks`. The syscalls are `set_priority(pid, prio)` (rejects values outside 1–20) and `get_priority(pid)` (returns −1 for unused or zombie processes, which `priotest` uses to detect completion).
- `scheduler()` scans all `NPROC` slots starting *after* the last process it ran and keeps the RUNNABLE process with the strictly lowest number. Starting the scan after the previous pick breaks ties in round-robin order. The chosen process gets `waitticks = 0`. Preemption is the existing per-tick `yield()`, so a higher-priority process takes the CPU within one tick.
- **Aging:** `clockintr()` on CPU 0 calls `agetick()`. Every RUNNABLE process gets `++waitticks`; once that passes 30, priority drops one number (minimum 1) and the counter resets.
- `priotest` (CPUS=1): the parent sets itself to priority 1 and forks 4 identical CPU-bound workers with priorities 2, 6, 12 and 20. It samples once per tick and is the only process that prints, so output never interleaves.

Order	PID	Initial priority	Priority when it finished	Completion tick
1	4	2	2	141
2	5	6	2	289
3	6	12	3	452
4	7	20	6	607





Completion order follows priority exactly. The priority-20 process never runs while better processes exist, but it ages 20→6 (one level every 31 ticks, visible as the staircase) and completes. Starvation is prevented. Process 5 caught up to priority 2 at t≈125 and then round-robined with process 4, which shows the tie-break.

Problems faced and how they were resolved

- **One trapframe address for all threads.** With a shared page table, two threads cannot both have their trapframe at `TRAPFRAME`. Fixed with the per-slot `THREADTRAP` mapping plus `sscratch` carrying the address into the trampoline.
- **Freeing a shared address space.** Without care, `freeproc()` of a thread would free the leader's pages, and a leader exiting first would leave threads on a freed page table (`freewalk: leaf` panic). Threads now only unmap their own trapframe page, and the leader kills and joins its threads in `exit()`. A test where the leader exits while two threads spin forever ran repeatedly without a panic.
- **sbrk cap broke `usertests`.** Lowering the heap limit below the thread-trapframe area failed `lazy_sbrk`. The lower cap now applies only while the address space is actually shared (`userlimit()`), and `clone` refuses to start if the heap already reaches that area. `usertests -q`: ALL TESTS PASSED.
- **Race did not appear at first.** Idle xv6 harts sit in `wfi` until their next timer tick, so all three threads ran one after another on the same hart and the counter came out exact by luck. A start barrier (`__sync_fetch_and_add(&ready,1)`, spin until 3) forces true parallel execution on CPUS=3.
- **Interleaved output.** xv6 `printf` writes one byte per `write()`. Output is printed only under MUTEX (prodcons) or from a single priority-1 monitor (priotest).
- **Workload too small.** 40M increments finished in about one tick, which shows no scheduling behaviour. Scaled to 6000×10^6 so each job takes roughly 145 ticks.